

OSSP-WEEK6-SKILL

SESSION11:

TASK1: Implement command history functionality in a shell. The program should store commands, maintain a history buffer, handle capacity limits, retrieve entries, display history, and validate consistency.

Source code:

```
GNU nano 7.2
#include <stdio.h>
#include <string.h>

#define MAX_HISTORY 5
#define MAX_COMMAND 100

char history[MAX_HISTORY][MAX_COMMAND];
int history_count = 0;

void add_history(char *command)
{
    if (history_count < MAX_HISTORY)
    {
        strcpy(history[history_count], command);
        history_count++;
    }
    else
    {
        for (int i = 0; i < MAX_HISTORY - 1; i++)
        {
            strcpy(history[i], history[i + 1]);
        }

        strcpy(history[MAX_HISTORY - 1], command);
    }
}

void show_history()
{
    printf("\nCommand History\n");
    printf("-----\n");

    if (history_count == 0)
    {
        printf("History is empty.\n");
        return;
    }

    for (int i = 0; i < history_count; i++)
    {
        printf("%d  %s\n", i + 1, history[i]);
    }
}

int main()
{

```

```

char command[MAX_COMMAND];

printf("Session 11 - Command History\n");
printf("-----\n");
printf("History capacity: %d commands\n", MAX_HISTORY);
printf("Use 'history' to display commands.\n");
printf("Use 'exit' to quit.\n\n");

while (1)
{
    printf("myshell> ");

    if (fgets(command, sizeof(command), stdin) == NULL)
        break;

    command[strcspn(command, "\n")] = '\0';

    if (strcmp(command, "exit") == 0)
    {
        printf("Shell exiting...\n");
        break;
    }

    if (strcmp(command, "history") == 0)
    {
        show_history();
        continue;
    }

    if (strlen(command) == 0)
        continue;

    add_history(command);
    printf("Command stored in history.\n");
}

return 0;
}

```

Output:

```

anuan@Anushka:~$ gcc history.c -o history
anuan@Anushka:~$ ./history

--- Shell History ---
1. Add command
2. Display history
3. Retrieve command
4. Check consistency
5. Exit
Enter choice: ls
Invalid choice.

--- Shell History ---
1. Add command
2. Display history
3. Retrieve command
4. Check consistency
5. Exit
Enter choice: Invalid choice.

--- Shell History ---
1. Add command
2. Display history
3. Retrieve command
4. Check consistency
5. Exit

```

Observation: I learned how to implement command history using a fixed-size history buffer. I understood how commands are stored, retrieved, and displayed, and how the oldest command is removed when the history capacity is reached.

Task2: Implement pipeline functionality in a shell. The program should create pipeline structures, store commands, maintain execution order, connect processes, validate the pipeline layout, and test configurations.

Source code:

```

GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    int pipefd[2];
    pid_t pid1, pid2;

    printf("Session 11 - Pipeline Structure\n");
    printf("-----\n");
    printf("Pipeline: ls -l | grep .c\n");

    if (pipe(pipefd) == -1)
    {
        perror("pipe");
        return 1;
    }

    printf("Pipeline created successfully.\n");
    printf("Execution order:\n");
    printf("1. ls -l\n");
    printf("2. grep .c\n");

    pid1 = fork();
    if (pid1 < 0)
    {
        perror("fork");
        return 1;
    }

    if (pid1 == 0)
    {
        close(pipefd[0]);
        dup2(pipefd[1], STDOUT_FILENO);
        close(pipefd[1]);

        execlp("ls", "ls", "-l", NULL);

        perror("execlp ls");
        exit(1);
    }

```

```

        perror("execlp ls");
        exit(1);
    }

    pid2 = fork();
    if (pid2 < 0)
    {
        perror("fork");
        return 1;
    }

    if (pid2 == 0)
    {
        close(pipefd[1]);
        dup2(pipefd[0], STDIN_FILENO);
        close(pipefd[0]);

        execlp("grep", "grep", ".c", NULL);

        perror("execlp grep");
        exit(1);
    }

    close(pipefd[0]);
    close(pipefd[1]);

    waitpid(pid1, NULL, 0);
    waitpid(pid2, NULL, 0);

    printf("\nPipeline execution completed successfully.\n");

    return 0;
}

```

Output:

```
anuan@Anushka:~$ nano pipeline.c
anuan@Anushka:~$ gcc pipeline.c -o pipeline
anuan@Anushka:~$ ./pipeline
Enter number of commands (max 5): 2
Enter command 1: ls
Enter command 2: wc -l

Pipeline Layout:
[1] ls
  | [2] wc -l

Pipeline is valid with 2 commands.
```

Observation: I learned how to implement a shell pipeline using `pipe()`, `fork()`, `dup2()`, and `exec`. I understood how the output of one process can be connected to the input of another process and how the parent process maintains the execution order using `waitpid()`.

Session 12

Task1: Implement a program to create pipes, redirect streams, execute connected commands, close descriptors, synchronize processes, and test data flow.

Source code:

```

GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    int pipefd[2];
    pid_t pid1, pid2;

    printf("Session 12 - Task 1: Pipe Data Flow\n");
    printf("-----\n");
    printf("Command: ls -l | grep .c\n");

    if (pipe(pipefd) == -1)
    {
        perror("pipe");
        return 1;
    }

    pid1 = fork();

    if (pid1 < 0)
    {
        perror("fork");
        return 1;
    }

    if (pid1 == 0)
    {
        // First process: ls -l
        close(pipefd[0]);

        dup2(pipefd[1], STDOUT_FILENO);

        close(pipefd[1]);

        execlp("ls", "ls", "-l", NULL);

        perror("execlp ls");
        exit(1);
    }

    pid2 = fork();

    if (pid2 < 0)

```

```

        perror("fork");
        return 1;
    }

    if (pid2 == 0)
    {
        // Second process: grep .c
        close(pipefd[1]);

        dup2(pipefd[0], STDIN_FILENO);

        close(pipefd[0]);

        execlp("grep", "grep", ".c", NULL);

        perror("execlp grep");
        exit(1);
    }

    // Parent closes unused descriptors
    close(pipefd[0]);
    close(pipefd[1]);

    // Wait for both processes
    waitpid(pid1, NULL, 0);
    waitpid(pid2, NULL, 0);

    printf("\nData flow completed successfully.\n");

    return 0;
}

```

Output:

```

anuan@anushka:~$ nano session12_task1.c
anuan@anushka:~$ gcc session12_task1.c -o session12_task1
anuan@anushka:~$ ./session12_task1
Session 12 - Task 1: Pipe Communication
-----
drwxr-xr-x 11 anuan anuan 4096 Aug 7 03:59 Practical
-rwxr-xr-x 1 anuan anuan 16424 Sep 12 03:44 client
-rw-r--r-- 1 anuan anuan 587 Sep 12 03:44 client.c
-rwxr-xr-x 1 anuan anuan 16400 Aug 8 03:24 command_executor
-rw-r--r-- 1 anuan anuan 1143 Aug 8 03:24 command_executor.c
-rw-r--r-- 1 anuan anuan 1988 Aug 25 04:55 double_quotes.c
-rw-r--r-- 1 anuan anuan 1990 Aug 25 04:57 double_quotes.c.save
-rwxr-xr-x 1 anuan anuan 16216 Aug 7 05:27 file_copy
-rw-r--r-- 1 anuan anuan 1113 Aug 7 05:27 file_copy.c
-rw-r--r-- 1 anuan anuan 0 Aug 7 05:25 file_copy.c.save
-rwxr-xr-x 1 anuan anuan 16880 Aug 5 03:06 fork_exec
-rw-r--r-- 1 anuan anuan 516 Aug 5 03:01 fork_exec.c
-rw-r--r-- 1 anuan anuan 517 Aug 5 03:04 fork_exec.c.save
-rw-r--r-- 1 anuan anuan 1299 Aug 5 03:05 fork_exec.r
-rw-r--r-- 1 anuan anuan 728 Aug 8 03:20 hardware_os.c
-rw-r--r-- 1 anuan anuan 151 Aug 17 04:11 hello.c
-rw-r--r-- 1 anuan anuan 2486 Sep 19 02:48 history.c
-rw-r--r-- 1 anuan anuan 1803 Aug 7 05:37 keyboard_input.c
-rw-r--r-- 1 anuan anuan 4239 Aug 11 06:43 lex_parser.c
-rw-r--r-- 1 anuan anuan 359 Aug 17 04:04 main_loop.c
-rw-r--r-- 1 anuan anuan 1109 Aug 24 09:19 p.c
-rw-r--r-- 1 anuan anuan 8864 Aug 11 06:34 parser.c
-rw-r--r-- 1 anuan anuan 8864 Aug 18 06:54 parser_lexer.c
-rw-r--r-- 1 anuan anuan 833 Aug 11 06:42 parser_lexer.c.save
-rw-r--r-- 1 anuan anuan 680 Aug 24 09:19 pc.c.save
-rw-r--r-- 1 anuan anuan 1113 Aug 24 09:39 pd.c
-rw-r--r-- 1 anuan anuan 1381 Aug 24 09:51 pi.c.save
-rwxr-xr-x 1 anuan anuan 16344 Aug 29 03:14 pipe_exec
-rw-r--r-- 1 anuan anuan 1164 Aug 29 03:14 pipe_exec.c
-rw-r--r-- 1 anuan anuan 3234 Sep 19 02:54 pipeline.c
-rw-r--r-- 1 anuan anuan 1161 Aug 24 09:50 pk.c
-rw-r--r-- 1 anuan anuan 685 Aug 24 09:47 plc.c
-rw-r--r-- 1 anuan anuan 800 Aug 24 09:33 pr.c.save
-rwxr-xr-x 1 anuan anuan 16296 Aug 17 04:08 process
-rw-r--r-- 1 anuan anuan 81 Aug 4 05:10 process.
-rw-r--r-- 1 anuan anuan 83 Aug 17 04:08 process.c
-rw-r--r-- 1 anuan anuan 797 Aug 4 05:35 process.c.save
-rw-r--r-- 1 anuan anuan 1497 Aug 24 08:59 process.c.save.1
-rwxr-xr-x 1 anuan anuan 16296 Aug 8 02:52 process_states
-rw-r--r-- 1 anuan anuan 1376 Aug 8 02:52 process_states.c
-rwxr-xr-x 1 anuan anuan 16400 Aug 29 03:10 producer_consumer
-rw-r--r-- 1 anuan anuan 1564 Aug 29 03:10 producer_consumer.c
-rw-r--r-- 1 anuan anuan 1388 Aug 24 09:04 product.c.save
-rw-r--r-- 1 anuan anuan 476 Aug 6 05:22 program.c
-rw-r--r-- 1 anuan anuan 965 Aug 24 09:25 ps.c
-rw-r--r-- 1 anuan anuan 1137 Aug 24 09:45 pt.c
-rw-r--r-- 1 anuan anuan 772 Sep 17 05:09 server.c
-rw-r--r-- 1 anuan anuan 1390 Sep 19 03:13 session12_task1.c
-rw-r--r-- 1 anuan anuan 800 Sep 12 03:46 signals.c
-rw-r--r-- 1 anuan anuan 872 Aug 25 04:47 single_quotes.c
-rw-r--r-- 1 anuan anuan 780 Aug 8 03:04 state_monitor.c
-rw-r--r-- 1 anuan anuan 726 Aug 28 05:17 task2.c
-rw-r--r-- 1 anuan anuan 723 Aug 28 05:15 task2.c.save
-rw-r--r-- 1 anuan anuan 1137 Aug 28 05:10 wait_waitpid.c
-rw-r--r-- 1 anuan anuan 987 Aug 18 06:52 waitpid_demo.c
Pipe execution completed successfully.
anuan@anushka:~$ |

```

Observation: I learned how to create a pipe and connect two processes using pipe(), fork(), and dup2(). I understood how the output of one command can be redirected as the input of another command and how unused file descriptors are closed to maintain proper data flow.

Task2: Implement a program to support multiple pipes, manage descriptor arrays, launch multiple processes, coordinate execution, clean up resources, and test long pipelines.

Source code:

```

GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

#define COMMANDS 3

int main()
{
    int pipes[COMMANDS - 1][2];
    pid_t pids[COMMANDS];

    printf("Session 12 - Task 2: Multiple Pipes\n");
    printf("-----\n");
    printf("Pipeline: ls -l | grep .c | wc -l\n\n");

    // Create pipes
    for (int i = 0; i < COMMANDS - 1; i++)
    {
        if (pipe(pipes[i]) == -1)
        {
            perror("pipe");
            return 1;
        }
    }

    // Create processes
    for (int i = 0; i < COMMANDS; i++)
    {
        pids[i] = fork();

        if (pids[i] < 0)
        {
            perror("fork");
            return 1;
        }

        if (pids[i] == 0)
        {
            // First command
            if (i == 0)
            {
                close(pipes[0][0]);

                dup2(pipes[0][1], STDOUT_FILENO);

```

```

                dup2(pipes[0][0], STDIN_FILENO);
                dup2(pipes[1][1], STDOUT_FILENO);
            }

            // Last command
            else if (i == 2)
            {
                dup2(pipes[1][0], STDIN_FILENO);
            }

            // Close all pipe descriptors
            for (int j = 0; j < COMMANDS - 1; j++)
            {
                close(pipes[j][0]);
                close(pipes[j][1]);
            }

            // Execute command
            if (i == 0)
                execlp("ls", "ls", "-l", NULL);

            else if (i == 1)
                execlp("grep", "grep", ".c", NULL);

            else
                execlp("wc", "wc", "-l", NULL);

            perror("execlp");
            exit(1);
        }
    }

    // Parent closes all pipe descriptors
    for (int i = 0; i < COMMANDS - 1; i++)
    {
        close(pipes[i][0]);
        close(pipes[i][1]);
    }

    // Wait for all children
    for (int i = 0; i < COMMANDS; i++)
    {
        waitpid(pids[i], NULL, 0);
    }

    printf("Long pipeline execution completed successfully.\n");

    return 0;
}

```

Output:

```
anun@anushka:~$ nano session12_task2.c
anun@anushka:~$ gcc session12_task2.c -o session12_task2
anun@anushka:~$ ./session12_task2
Session 12 - Task 2: Multiple Pipes
-----
Pipeline: ls -l | grep .c | wc -l

48
Long pipeline execution completed successfully.
anun@anushka:~$ |
```

Observation: I learned how to implement multiple pipes for a long command pipeline. I understood how to manage multiple file descriptors, create and coordinate multiple processes, connect their input and output streams, and clean up resources after execution.